

Project - High Level Design

On

Multimodal Manufacturing Creator

Course Name: Generative Ai

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Aaditya Shrivastava	EN22CS301007
2	Akshita Bhawsar	EN22CS301098
3	Akanksha Sharma	EN22CS301080
4	Aakanksha Jaiswal	EN22CS301011
5	Abhishek Malviya	EN22CS301034

Group Name: Group 12D1

Project Number: GAI-12

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ajaj Khan

*Academic Year:*2025-2026

1. Introduction

The Multimodal Manufacturing Creator is a GenAI-powered application designed to bridge the gap between conceptual manufacturing ideas and their visual and textual representations. The system integrates a Large Language Model (LLM) with an image generation model to translate user-provided text prompts into both detailed descriptive narratives and high-fidelity visual prototypes.

This project is developed as part of the Datagami Skill Based Course at Medicaps University. It demonstrates the practical application of multimodal AI in the manufacturing domain, combining state-of-the-art NLP and computer vision techniques to create a comprehensive content generation pipeline.

1.1 Purpose

The purpose of this document is to provide a High Level Design (HLD) overview of the Multimodal Manufacturing Creator application. It describes the system architecture, component design, data flow, API integrations, and key design decisions made during the development process.

1.2 Scope of the Document

This document covers the following aspects of the system:

- System architecture and design overview
- Application workflow and process flow
- Component-level design of all modules
- API catalogue and integration details
- Data design including storage and retrieval mechanisms
- Non-functional requirements including security and performance
- Key design considerations and trade-offs

1.3 Intended Audience

This document is intended for the following stakeholders:

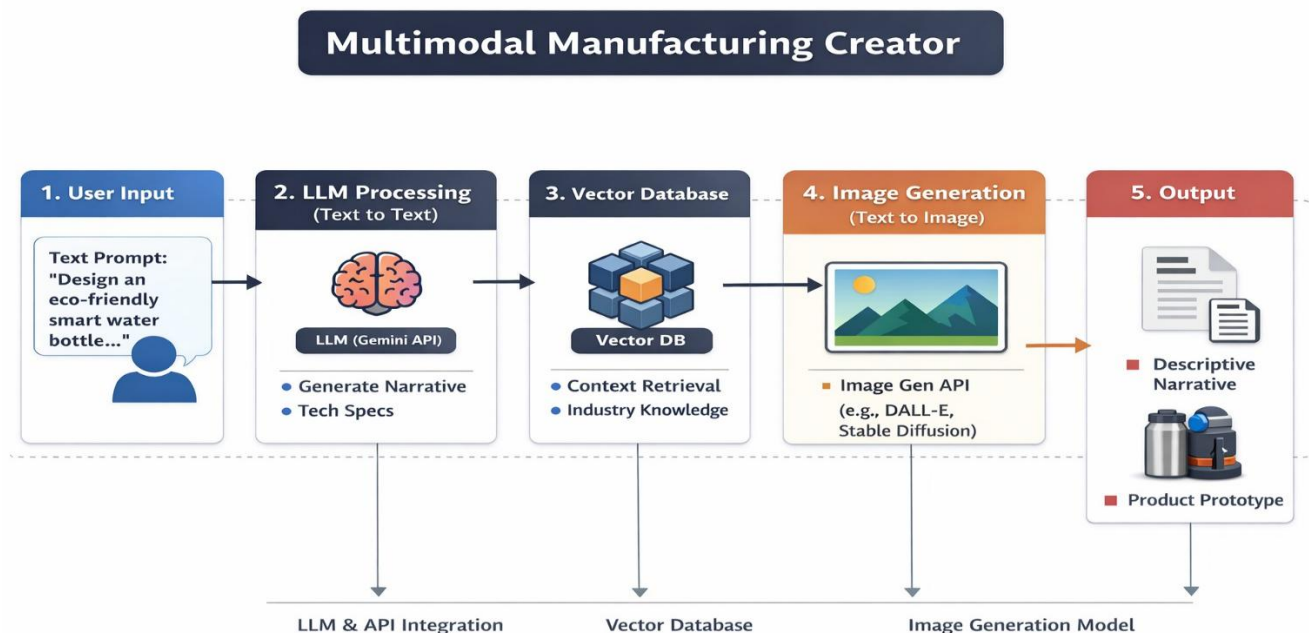
- University mentors and evaluators reviewing the project
- Industry mentors providing technical guidance
- Development team members working on the project
- Future developers who may extend or maintain the system

2. System Overview

The Multimodal Manufacturing Creator is a Python-based web application that accepts natural language descriptions of manufacturing concepts and produces two complementary outputs: a structured textual narrative and a photorealistic visual representation. The system leverages OpenAI's GPT-4o-mini for text generation and DALL·E 3 for image generation, with ChromaDB serving as a local vector database to enable semantic search and retrieval of previously generated concepts.

Component	Description
Frontend	Streamlit web interface with professional UI, tabs, and sidebar controls
LLM Module	GPT-4o-mini for generating manufacturing narratives and optimized image prompts
Image Module	DALL·E 3 for generating high-fidelity visual representations of concepts
Vector DB	ChromaDB (local) for storing and retrieving concept embeddings
Embeddings	OpenAI text-embedding-ada-002 for semantic vector representations

3. System Design



Step 1: User Input Layer

The workflow begins when the user provides a manufacturing idea in natural language format.

For example: “Design an eco-friendly smart water bottle for industrial workers.”

This input is captured by the FastAPI backend and passed to the AI processing layer.

The prompt becomes the foundation for both:

- Narrative generation
- Image generation

Step 2: LLM Processing (Text-to-Text)

The prompt is processed using the GPT-4o-mini(LLM) Large Language Model.

The LLM converts the idea into: a) A structured product description

b) Technical specifications

c) Material suggestions

d) Use-case explanation

This stage transforms an abstract idea into meaningful engineering documentation.

Step 3: Data Storage Layer(Vector Database)

LLM alone gives general answers. Vector DB enables Retrieval-Augmented Generation (RAG).

The Vector Database enhances the intelligence of the system by retrieving domain-specific knowledge. This ensures that the generated manufacturing concept is context-aware and industry-relevant.

Step 4: Image Generation Layer

The structured description is then passed to the image generation Model (DALL-E3)

The architecture is designed to integrate real text-to-image models such as Stable Diffusion or DALL·E.

It generates:

- Prototype visualization
- Concept design image.

Step 5: Output Layer

Finally, the system returns both:

- A descriptive narrative
- A product prototype visualization

3.1 Application Design

The application follows a modular, layered architecture with clear separation of concerns. Each functional area is encapsulated in its own module, making the system easy to maintain, test, and extend. The frontend (Streamlit) communicates with backend modules through direct Python function calls.

Architecture layers:

- Presentation Layer: Streamlit UI (app.py) — handles user interaction and displays results
- Business Logic Layer: llm_handler.py, image_handler.py — core generation logic
- Data Layer: vector_db.py — handles persistence, retrieval, and similarity search
- Configuration Layer: config.py — centralizes all settings and API key management

3.2 Process Flow

The following steps describe the end-to-end process when a user submits a manufacturing concept:

1. User enters a manufacturing concept prompt in the Streamlit UI
2. The system searches ChromaDB for similar previously generated concepts
3. GPT-4o-mini generates a structured narrative (description + materials)
4. GPT-4o-mini optimizes the prompt for DALL·E 3 image generation
5. DALL·E 3 generates a photorealistic image of the manufacturing concept
6. Image is saved locally to data/stored_concepts/ folder
7. Narrative and metadata are embedded and stored in ChromaDB
8. Results are displayed in two-column layout with download options

3.3 Information Flow

Data flows through the system in the following sequence:

Source	Data	Destination	Format
User (UI)	Text Prompt	llm_handler.py	String
llm_handler.py	Narrative Text	UI + vector_db.py	String
llm_handler.py	Optimized Prompt	image_handler.py	String
image_handler.py	Image URL + File	UI + Local Disk	PNG / URL
vector_db.py	Embeddings	ChromaDB	Float Vector
ChromaDB	Similar Concepts	UI	JSON

4. Components Design

4.1 app.py — Streamlit Frontend

The main application file that serves as the entry point. It handles all user interactions and orchestrates calls to the backend modules.

- Renders the two-tab layout: Generate Concept and History Gallery
- Sidebar controls for max tokens, image quality, and toggle settings
- Progress bar providing real-time feedback during generation
- Stat cards showing total concepts, model names, and token settings
- Download buttons for both narrative (.txt) and image (.png)

4.2 llm_handler.py — LLM Module

Handles all interactions with the OpenAI GPT-4o-mini model. Responsible for two distinct generation tasks:

- `generate_narrative()`: Produces a structured description with materials section using a focused system prompt
- `generate_image_prompt()`: Creates an optimized, visually-rich prompt for DALL·E 3 based on the narrative context

4.3 image_handler.py — Image Generation Module

Manages DALL·E 3 API calls and local image persistence.

- `generate_image()`: Calls DALL·E 3 API with the optimized prompt
- Supports standard and HD quality settings configurable from the sidebar

- `_save_image()`: Downloads and saves generated images with timestamped filenames to `data/stored_concepts/`
- Returns both the temporary URL and the local file path

4.4 `vector_db.py` — Vector Database Module

Manages all ChromaDB operations for persistent storage and semantic search.

- `store_concept()`: Embeds and stores user prompt, narrative, and image metadata
- `search_similar()`: Performs semantic similarity search using cosine distance
- `get_all_concepts()`: Retrieves full history for the History Gallery tab
- Uses OpenAI text-embedding-ada-002 for generating vector embeddings

4.5 `config.py` — Configuration Module

Centralizes all application settings and API key management.

- Loads environment variables from `.env` file using `python-dotenv`
- Defines model names, token limits, image size, and quality settings
- Provides fallback values to prevent runtime crashes on missing config
- Single source of truth for all configurable parameters

5. Key Design Considerations

5.1 Single API Provider Strategy

The system uses OpenAI exclusively for LLM, image generation, and embeddings. This reduces complexity, requires only one API key, and simplifies billing. The trade-off is vendor dependency, but this is acceptable for an educational project scope.

5.2 Two-Step Image Generation

Rather than passing user prompts directly to DALL·E 3, the system first uses GPT-4o-mini to optimize the prompt for image generation. This produces significantly better visual outputs by adding photorealistic detail, material descriptions, and professional settings that raw user prompts typically lack.

5.3 Local Vector Database

ChromaDB runs locally with no external service dependency. This eliminates the need for a cloud vector DB subscription, works offline, and keeps all data on the user's machine. The persistent client ensures data survives application restarts.

5.4 Modular Architecture

Each concern is isolated in its own module (llm_handler, image_handler, vector_db, config). This allows individual components to be swapped or upgraded independently. For example, switching from DALL·E 3 to Stable Diffusion would only require changes in image_handler.py.

5.5 Cost Optimization

GPT-4o-mini was chosen over GPT-4o due to its significantly lower cost (~20x cheaper) while maintaining sufficient quality for manufacturing concept descriptions. The focused system prompt also limits output length, further reducing token consumption per request.

6. API Catalogue

API	Provider	Model	Purpose	Approx. Cost
Chat Completions	OpenAI	gpt-4o-mini	Narrative + prompt optimization	~\$0.001/req
Image Generation	OpenAI	dall-e-3	Visual prototype generation	~\$0.04/image
Embeddings	OpenAI	text-embedding-ada-002	Vector embeddings for search	~\$0.00/req
Vector Store	ChromaDB	Local	Concept storage and retrieval	Free (local)

7. Data Design

7.1 Data Model

Each concept stored in ChromaDB contains the following fields:

Field	Type	Description
id	String	Unique concept ID (e.g. concept_20260220_120000)
embedding	Float[]	768-dim vector from text-embedding-ada-002
document	String	Full generated narrative text
user_prompt	String (meta)	Original user input text
image_path	String (meta)	Local file path to saved PNG image
created_at	String (meta)	ISO 8601 timestamp of creation

7.2 Data Access Mechanism

Data is accessed through ChromaDB's Python client using three primary operations:

- Write: `collection.add()` — stores embeddings, documents, and metadata on concept creation
- Search: `collection.query()` — performs approximate nearest-neighbor search using cosine similarity
- Read All: `collection.get()` — retrieves all stored concepts for the History Gallery view

7.3 Data Retention Policies

All data is stored locally on the user's machine in the `./data/chroma_db` directory. There is no automatic expiration or deletion. Users may manually delete the `chroma_db` folder to reset the knowledge base. Generated images are stored in `./data/stored_concepts/` with timestamped filenames.

7.4 Data Migration

As this is a local application with no remote database, data migration is handled by copying the `./data/` directory between machines. No formal migration scripts are required for the current version.

8. Interfaces

8.1 User Interface

The application is served through Streamlit at `http://localhost:8501`. The UI consists of:

- Header Banner: Application title and description with gradient background
- Stats Row: Four cards showing concepts count, model names, and token settings
- Generate Concept Tab: Text input, generate button, progress bar, and two-column results

- History Gallery Tab: Chronological list of all past concepts with expandable narratives and images
- Sidebar: Generation settings, cost estimates, and model information

8.2 State and Session Management

Streamlit's session state (`st.session_state`) is used to persist UI preferences within a browser session. Long-term data persistence is handled by ChromaDB, which maintains state across application restarts through its `PersistentClient`. No user authentication or multi-session management is implemented in the current version.

8.3 Caching

The current version does not implement explicit caching. Future versions could use Streamlit's `st.cache_data` decorator to cache frequently accessed ChromaDB results and reduce latency. OpenAI API responses are not cached to ensure fresh results for each unique prompt.

9. Non-Functional Requirements

9.1 Security Aspects

- API Key Management: The OpenAI API key is stored in a local `.env` file and loaded at runtime. It is never hardcoded in source files committed to version control.
- No Authentication: The current version is designed for local use only. No user authentication is implemented.
- .gitignore Protection: The `.env` file must be listed in `.gitignore` to prevent accidental exposure of API keys in public repositories.
- Data Privacy: All user prompts and generated content are stored locally. No data is sent to external services beyond the OpenAI API calls.

9.2 Performance Aspects

- LLM Response Time: GPT-4o-mini typically responds in 2-5 seconds for narrative generation at 512-1024 tokens.
- Image Generation Time: DALL·E 3 typically takes 10-20 seconds to generate a 1024x1024 standard quality image.
- Embedding Generation: `text-embedding-ada-002` responds in under 1 second for typical text inputs.
- ChromaDB Search: Local vector similarity search completes in milliseconds for collections up to several thousand concepts.
- Progress Feedback: A real-time progress bar is displayed during generation to keep users informed of the current step.

10. References

- OpenAI API Documentation — <https://platform.openai.com/docs>
- OpenAI DALL·E 3 Guide — <https://platform.openai.com/docs/guides/images>
- ChromaDB Documentation — <https://docs.trychroma.com>
- Streamlit Documentation — <https://docs.streamlit.io>
- Python-dotenv Library — <https://pypi.org/project/python-dotenv>
- Medicaps University — <https://www.medicaps.ac.in>